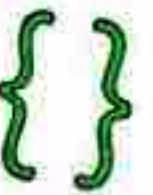
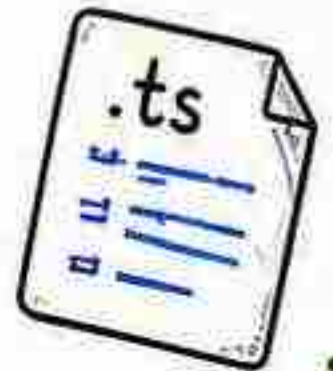


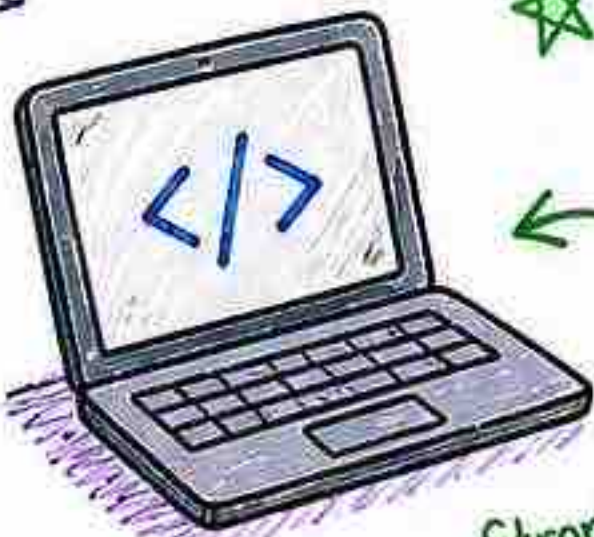


# TypeScript

## Basic



```
{ let name: string;  
  let age: number = 20;  
  let isDev: boolean =  
    true;  
}
```



Strong Types,  
Better Code!

(10 Days)

Learn TypeScript  
step by step in  
just 10 Days!

Type Safe  
JavaScript!



Day 1: Introduction to TypeScript & Environment Setup

Day 2: Variables, Data Types, and Type Annotations

Day 3: Operators and Type Inference

Day 4: Functions and Return Types

Day 5: Arrays and Tuples

Day 6: Objects and Type Aliases

Day 7: Interfaces Fundamentals

Day 8: Conditional Statements and Loops

Day 9: Modules and File Organization

Day 10: Mini Project - Student Management System

Code Today,  
Build Tomorrow!



console.log("Let's build awesome things! 🚀");





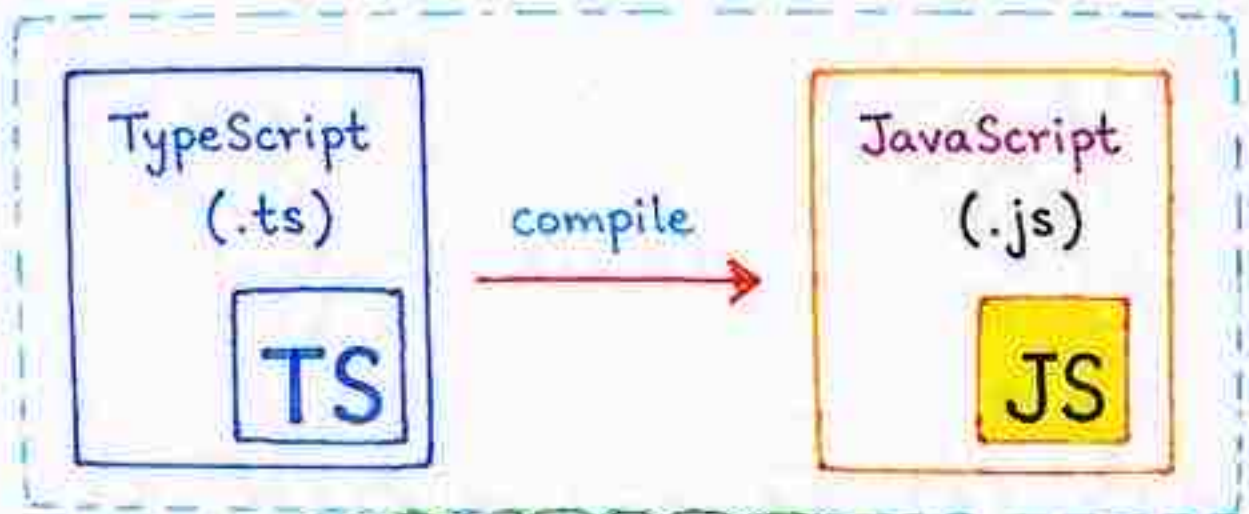
# TypeScript

## BASIC (10 DAYS)

### DAY 1: Introduction to TypeScript & Environment Setup

#### ★ What is TypeScript?

- TypeScript is a **superset** of JavaScript.
- It adds **static typing** to JavaScript.
- Developed and maintained by **Microsoft**.
- Helps catch **errors early** and improves code quality.
- Compiles to **plain JavaScript**.



Write TypeScript today,  
Run JavaScript anywhere!

#### ★ Why TypeScript?

- ✓ Catches errors at compile time
- ✓ Better code completion & IntelliSense
- ✓ Easier refactoring
- ✓ Improved code readability & maintainability
- ✓ Great tooling support

#### ★ Key Features

- Static Typing
- Interfaces & Types
- Classes & OOP Support
- Generics
- Modern JavaScript Features



#### ★ Environment Setup

##### ① Install Node.js

Download and install from  
<https://nodejs.org>

##### Check Installation

- ✓ `node -v`
- ✓ `npm -v`

##### ② Install TypeScript

```
npm install -g typescript
```

##### Check Installation

- ✓ `tsc -v`

##### ③ Create Your First TypeScript File

- Create a file: `app.ts`
- Write some TypeScript code.
- Compile it using the TypeScript compiler.
- Run the generated JavaScript file.

app.ts

```
let message: string = "Hello TypeScript!";  
console.log(message);
```

Compile  
`tsc app.ts`

app.js

```
var message = "Hello TypeScript!";  
console.log(message);
```

#### ★ Summary

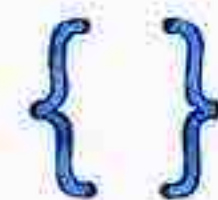
TypeScript helps us write better, safer and more maintainable JavaScript code.  
Let's start our TypeScript journey! 😊

Practice Makes Perfect! ❤️





# TypeScript



BASIC (10 DAYS)

## DAY 2: Variables, Data Types, and Type Annotations

### What are Variables?

- Variables are used to store data values.
- In TypeScript, we can specify the **type** of data a variable can hold.



### Declaring Variables

We use `let`, `const` or `var` to declare variables.

```
let name: string = "Alice";  
const age: number = 20;  
var isStudent: boolean = true;
```

Type Annotation

### Common Data Types

**string**

Textual data

```
let name: string  
= "Hello";
```

**number**

Numeric values

```
let age: number  
= 25;
```

**boolean**

True or False

```
let isActive: boolean  
= false;
```

**any**

Any type of value

```
let value: any = 10;  
value = "Hello";
```

**null**

Represents no value

```
let data: null = null;
```

**undefined**

Variable not assigned

```
let x: undefined =  
undefined;
```

### Type Annotations

- Type annotations let us explicitly define the type of a variable.
- This helps TypeScript catch errors at compile time.

```
let city: string = "New York";  
let score: number = 95.5;  
let passed: boolean = true;
```

: Type Annotation



### Note:

If we don't provide a type, TypeScript tries to **infer** it automatically.

### Quick Examples

```
let fullName: string = "John Doe";  
let height: number = 5.9;  
let isLoggedIn: boolean = false;  
let anything: any = "TypeScript";  
anything = 123;  
let nothing: null = null;  
let notAssigned: undefined = undefined;
```

### Type Inference vs Type Annotation

| Type Inference                                                                                                         | Type Annotation                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>TypeScript automatically detects the type.</li><li>Less code to write.</li></ul> | <ul style="list-style-type: none"><li>We explicitly define the type.</li><li>More clear &amp; better for maintainability.</li></ul> |

#### Inference Example

```
let message = "Hello";  
// TypeScript infers  
message: string
```

#### Annotation Example

```
let message: string = "Hello";  
// Type is explicitly  
defined
```

### Key Takeaway

Variables store data.  
Data types tell TypeScript what kind of data a variable can hold.  
Type annotations make our code more predictable and error-free!

Practice  
Today,  
Master  
Tomorrow!







# TypeScript

BASIC (10 DAYS)

## DAY 3: Operators and Type Inference

### ★ Operators in TypeScript

TypeScript supports all JavaScript operators.

| Category   | Operators                  | Example                                   | Result |
|------------|----------------------------|-------------------------------------------|--------|
| Arithmetic | + - * / % **               | let a = 10, b = 3;<br>a + b               | 13     |
| Assignment | = += -= ** /= %=           | let x = 5;<br>x += 2                      | 7      |
| Comparison | == === != !==<br>> < >= <= | let p = 5; p > 3                          | true   |
| Logical    | &&    !                    | true && false                             | false  |
| Bitwise    | &   ^ ~ << >>              | 5 & 1                                     | 1      |
| Others     | ?: (ternary)               | let age = 18;<br>age >= 18 ? 'Yes' : 'No' | 'Yes'  |

### ★ Type Inference

TypeScript can automatically infer the type of a variable based on its value.

```
let name = "Alice"; // inferred as string
let age = 25; // inferred as number
let isActive = true; // inferred as boolean
let scores = [10, 20, 30]; // inferred as number[]
let user = { id: 1, name: "Bob" };
// inferred as { id: number; name: string }
```



Type inference makes the code cleaner and reduces the need to explicitly write types everywhere.

### ★ Examples with Operators

```
let a = 10;
let b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000

console.log(a > b); // true
console.log(a === 10); // true
console.log(a != b); // true

let isLoggedIn = true;
console.log(isLoggedIn && a > b); // true
console.log(!isLoggedIn || b < a); // true

let max = (a > b) ? a : b; // 10
```

### ★ More on Type Inference

TypeScript infers types in different scenarios.

```
let message = "Hello TypeScript"; // string
let count = 100; // number
let flag = false; // boolean
let list = [1, 2, 3]; // number[]
let names = ["Tom", "Jerry"]; // string[]
let person = { name: "Tom", age: 20 }; // { name: string; age: number }

let anything = null; // null
let nothing = undefined; // undefined
```

#### ★ Note:

If a variable is declared without assigning a value, and without a type annotation, TypeScript infers it as 'any' (unless 'noImplicitAny' is enabled).

### Key Takeaway

- ✓ Operators work the same as in JavaScript.
- ✓ TypeScript adds type safety on top.
- ✓ Type inference automatically detects types, making code cleaner and easier to maintain.



### Practice Time!

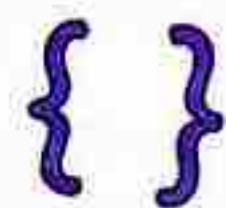
- 1 Declare variables using different values and check inferred types using your editor.
- 2 Write expressions using arithmetic, comparison and logical operators.
- 3 Try changing values and observe the results.







# TypeScript



## BASIC (10 DAYS)



### DAY 4: Functions and Return Types

#### ★ What is a Function?

- A function is a block of code designed to perform a task.
- Functions help us write **reusable** and organized code.
- In TypeScript, we can specify the **types** of **parameters** and **return values**.



#### ★ Function Syntax

```
function functionName(param1: Type, param2: Type): ReturnType {  
  // code to be executed  
  return value;  
}
```

Parameters (with types)      Return Type

#### ★ Example 1: Function with Return Type

```
function add(a: number, b: number): number {  
  return a + b;  
}  
  
let result: number = add(5, 3); // 8
```

Return type  
is **number**

#### ★ Example 2: Function with String Return

```
function greet(name: string): string {  
  return "Hello, " + name + "!";  
}  
  
let message: string = greet("Alice");  
// "Hello, Alice!"
```

Return type  
is **string**

#### ★ Example 3: Function with No Return (void)

```
function printMessage(msg: string): void {  
  console.log(msg);  
}  
  
printMessage("TypeScript is awesome!");  
// No return value
```

void means  
no value  
is returned

#### ★ Example 4: Function with Boolean Return

```
function isEven(n: number): boolean {  
  return n % 2 === 0;  
}  
  
let even: boolean = isEven(10); // true  
let odd: boolean = isEven(7);  // false
```

Return type  
is **boolean**

#### ★ Key Points

- ✓ Always specify the return type after the parameters.
- ✓ TypeScript uses the return type to check correctness.
- ✓ If a function does not return anything, use **void**.
- ✓ Consistent return types make code more predictable.
- ✓ TypeScript will give an error if the return type does not match.



#### ★ Quick Examples

```
function square(n: number): number {  
  return n * n;  
}  
  
function getFirstChar(str: string): string {  
  return str.charAt(0);  
}  
  
function sayHi(): void {  
  console.log("Hi there!");  
}
```

#### Remember:

A function's return type tells TypeScript what kind of value it will return.



#### Common Return Types

**number**

For numbers

**string**

For text

**boolean**

For true/false

**void**

For no return  
value

**any**

For any type  
of value

**Custom Type /  
Interface**

For complex  
data



#### Practice for Today

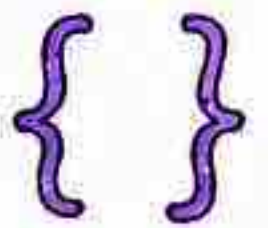
- ★ Write a function to find the area of a rectangle.
- ★ Write a function that returns the longer of two strings.
- ★ Write a function that prints your name (void).







# TypeScript



BASIC (10 DAYS)

## DAY 5: Arrays and Tuples

### ★ What is an Array?

- An array is a collection of items of the **same type** stored in a single variable.
- Arrays are ordered and **zero-indexed**.

```
[ 10, 20, 30, 40 ]
 |   |   |   |
 0   1   2   3
```



### ★ What is a Tuple?

- A tuple is an array with a **fixed number** of elements where each element can be of a **different type**.
- Tuples help represent a fixed structure.

```
[ "Alice", 25, true ]
   ^       ^       ^
   string  number  boolean
```

### ★ Arrays in TypeScript

#### ① Declare an Array

```
// Using type annotation
let numbers: number[] = [10, 20, 30];
let names: string[] = ["Tom", "Jerry"];
let flags: boolean[] = [true, false, true];
let anyItems: any[] = [1, "hello", true];
```

#### ② Initialize an Array

```
let scores: number[] = []; // empty array
let cities = ["New York", "London"]; // type inferred
```

#### ③ Access Array Elements

```
let nums: number[] = [5, 10, 15, 20];
console.log(nums[0]); // 5
console.log(nums[2]); // 15
```

#### ④ Update Elements

```
nums[1] = 25; // [5, 25, 15, 20]
```

#### ⑤ Array with Type Inference

```
let fruits = ["Apple", "Banana", "Mango"];
// TypeScript infers: string[]
```

All elements must be of the same type.



### ★ Tuples in TypeScript

#### ① Declare a Tuple

```
let user: [string, number, boolean] = ["Alice", 25, true];
let point: [number, number] = [10, 20];
```

#### ② Access Tuple Elements

```
console.log(user[0]); // Alice
console.log(user[1]); // 25
console.log(user[2]); // true
```

#### ③ Tuple with Optional Elements

```
let employee: [number, string, number?] = [1, "Sam"];
// number (id), string (name), number? (salary)
employee.push(50000); // [1, "Sam", 50000]
```

#### ④ Readonly Tuple

```
let coords: readonly [number, number] = [0, 0];
// coords[0] = 5; // ✗ Error: cannot assign
// to '0' because it is a read-only property.
```

Each position has a specific type.



### ★ Key Differences

|             | Arrays                | Tuples                 |
|-------------|-----------------------|------------------------|
| Elements    | Same type             | Can be different types |
| Size        | Can grow or shrink    | Fixed length           |
| Declaration | type[]                | [type1, type2, ...]    |
| Use case    | List of similar items | Fixed structure data   |

### ★ Common Array Methods

```
push() → Adds an item at the end
pop() → Removes last item
shift() → Removes first item
unshift() → Adds item at the beginning
length → Returns number of elements
```

#### Note:

Tuples have limited methods compared to arrays.



### Practice Time! ✨

- ① Create an array of numbers and find the sum of all elements.
- ② Create a tuple to store: [id, name, isActive].
- ③ Swap the values of two variables using a tuple.



Tuples are great for representing records like coordinates, RGB values, database rows, etc.





TS

# TypeScript

BASIC (10 DAYS)

## DAY 6: Objects and Type Aliases

### Objects in TypeScript

- ★ Objects store **key-value** pairs.
- ★ Keys are **strings** (or **Symbols**).
- ★ Values can be any data type.

•• Example ••

```
let person = {
  name: "John",
  age: 22,
  isStudent: true
};

console.log(person.name); // John
```



Access  
using  
dot (.)

### Type Aliases

- ✓ A type alias gives a name to a type.
- ✓ It helps to reuse complex types.
- ✓ Created using **'type'** keyword.

•• Example ••

```
type Person = {
  name: string;
  age: number;
  isStudent: boolean;
};

let p: Person = {
  name: "Alice",
  age: 20,
  isStudent: false
};
```

### Why use Type Aliases?

- ✓ Better readability
- ✓ Reusability
- ✓ Maintainability
- ✓ Avoid duplicate type definitions

### Objects - More Examples

#### 1. Nested Object

```
let user = {
  id: 101,
  name: "Bob",
  address: {
    city: "Chennai",
    pin: 600001
  }
};
```

Nested object

#### 2. Object with Methods

```
let calc = {
  a: 10,
  b: 5,
  add() {
    return this.a + this.b;
  }
};

console.log(calc.add()); // 15
```

### Type Alias with Complex Types

#### a) Alias for Object

```
type Product = {
  id: number;
  name: string;
  price: number;
  inStock: boolean;
};

let prod: Product = {
  id: 1,
  name: "Laptop",
  price: 55000,
  inStock: true
};
```

#### b) Alias for Union Type

```
type Status = "Pending" |
              "Completed" |
              "Cancelled";

let orderStatus: Status =
  "Pending";
```

### Note

Type aliases can be used with unions, intersections, tuples and more!

### Differences: Object vs Type Alias

#### Objects

- ✓ Actual value exists at runtime.
- ✓ Declared with **let**, **const**.
- ✓ Used to store data.
- ✓ Example: **let user = {...}**

#### Type Aliases

- ✓ Only a type, no runtime value.
- ✓ Declared with **type**.
- ✓ Used to define types.
- ✓ Example: **type User = {...}**

### Quick Examples

#### Update Object

```
person.age = 23;
person.name = "Mike";
```

#### Add New Property

```
person.city = "Pune";
// Allowed
// (extra property)
```

#### Readonly-like (by alias)

```
type ReadonlyPerson = {
  readonly id: number;
  readonly name: string;
};
```

### Key Takeaways

- ✓ Objects hold data as key-value pairs.
- ✓ Type aliases give names to types.
- ✓ They help write clean, reusable and maintainable code.



### Practice Time!

- ① Create an object for a student (name, age, grade).
- ② Create a type alias for Address and use it in a Person.
- ③ Use a type alias for union type: **"Admin" | "User" | "Guest"**.
- ④ Create an object with nested objects and update values.





# TypeScript

BASIC (10 DAYS)

## DAY 7: Interfaces Fundamentals

### ★ What is an Interface?

- An interface is a blueprint of an object.
- It defines the shape of an object.
- It contains properties and their types.
- It helps in type checking and code consistency.



### ★ Why use Interfaces?

- ✓ Define the structure of objects.
- ✓ Improve code readability and maintainability.
- ✓ Enable type safety.
- ✓ Help in collaboration and large-scale development.

Think of it as a contract that an object must follow.



### ★ 1. Basic Interface Syntax

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  isActive: boolean;  
}
```

Name of the interface

Properties with types

No semicolon (;) after the interface block. 😊

### ★ 2. Using an Interface

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  isActive: boolean;  
}  
  
let user: User = {  
  id: 1,  
  name: "Alice",  
  email: "alice@example.com",  
  isActive: true  
};
```

Define the structure

Object must follow the interface structure

### ★ Optional Properties

```
interface Product {  
  id: number;  
  name: string;  
  price: number;  
  description?: string; // optional  
}  
  
let p: Product = {  
  id: 101,  
  name: "Pen",  
  price: 20  
  // description is optional  
};
```

Use ? to make a property optional.

### ★ Readonly Properties

```
interface Student {  
  readonly id: number; // cannot be changed  
  name: string;  
  grade: number;  
}  
  
let s: Student = {  
  id: 1,  
  name: "John",  
  grade: 90  
};  
  
// s.id = 2; // ❌ Error: Cannot assign to 'id'
```

Use readonly to make a property read-only.



### ★ Interface with Function Type

```
interface Calculator {  
  add(a: number, b: number): number;  
  subtract(a: number, b: number): number;  
}  
  
let calc: Calculator = {  
  add: (a, b) => a + b,  
  subtract: (a, b) => a - b  
};
```

Interfaces can also define methods.

### ★ Interface with Array

```
interface Book {  
  id: number;  
  title: string;  
  author: string;  
}  
  
let books: Book[] = [  
  { id: 1, title: "TS Guide", author: "Max" },  
  { id: 2, title: "Node Basics", author: "Sam" }  
];
```

Use interface with arrays of objects.



### ★ --- Key Takeaways ---

- ✓ Interfaces define the structure of an object.
- ✓ They help in writing clean, predictable and maintainable code.
- ✓ Use ?, readonly and methods in interfaces to make them more powerful.



### Common Example

```
interface Employee {  
  id: number;  
  name: string;  
  role: string;  
  salary?: number;  
}  
  
let emp: Employee = {  
  id: 10,  
  name: "Priya",  
  role: "Developer",  
  salary: 50000  
};
```



### Practice Time! ✨

- ① Create an interface for Car with properties: brand, model, year.
- ② Create an object that follows the interface.
- ③ Add an optional property 'color'.
- ④ Try making one property readonly. 😊





# TypeScript

BASIC (10 DAYS)

## DAY 8: Conditional Statements and Loops

### Why Important?

- Conditional statements help our program make **decisions**.
- Loops help us **repeat actions** efficiently.



### Conditional Statements

Used to execute code based on certain conditions.

#### 1. if Statement

```
let age: number = 18;
if (age >= 18) {
  console.log("You are eligible to vote.");
}
```

#### 2. if...else Statement

```
let isRaining: boolean = false;
if (isRaining) {
  console.log("Take an umbrella.");
} else {
  console.log("Enjoy your day!");
}
```

#### 3. if...else if...else Ladder

```
let marks: number = 85;
if (marks >= 90) {
  console.log("Grade: A");
} else if (marks >= 75) {
  console.log("Grade: B");
} else if (marks >= 60) {
  console.log("Grade: C");
} else {
  console.log("Grade: D");
}
```

TypeScript uses the same conditional syntax as JavaScript!



## Loops

Used to repeat a block of code multiple times.

#### 1. for Loop

```
for (let i = 1; i <= 5; i++) {
  console.log("Count: " + i);
}
```

##### Output:

Count: 1  
Count: 2  
Count: 3  
Count: 4  
Count: 5

#### 2. while Loop

```
let i: number = 1;
while (i <= 5) {
  console.log("Count: " + i);
  i++;
}
```

##### Output:

Count: 1  
Count: 2  
Count: 3  
Count: 4  
Count: 5

#### 3. do...while Loop

```
let i: number = 1;
do {
  console.log("Count: " + i);
  i++;
} while (i <= 5);
```

##### Output:

Count: 1  
Count: 2  
Count: 3  
Count: 4  
Count: 5

#### 4. for...of Loop

```
let fruits: string[] = ["Apple", "Banana", "Mango"];
for (let fruit of fruits) {
  console.log(fruit);
}
```

##### Output:

Apple  
Banana  
Mango



### Break and Continue

**break:** exits the loop

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) break;
  console.log(i);
}
```

##### Output:

1  
2

**continue:** skips current iteration

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i);
}
```

##### Output:

1  
2  
4  
5

### Quick Notes

- ✓ Use **if...else** to make decisions.
- ✓ Use **loops** to repeat code.
- ✓ Choose the right loop based on the need.
- ✓ **for:** when count is known
- ✓ **while:** when condition is unknown.



### Practice Time!

- 1 Create a program to check if a number is positive, negative or zero.
- 2 Print numbers from 1 to 10 using a for loop.
- 3 Print even numbers from 1 to 20 using a while loop.
- 4 Use break to stop the loop when number is 5.



### Key Takeaway

Conditional statements help our program make smart decisions.  
Loops help us automate repetitive tasks and save time!







# TypeScript { }

BASIC (10 DAYS)

## DAY 9: Modules and File Organization

### Why Modules?

- ✓ Break code into separate files.
- ✓ Reusability of code.
- ✓ Better maintainability.
- ✓ Avoid polluting the global scope.



### What is a Module?

A module is a file that contains code and can **export** values for use in other files.

Every file with **import** or **export** is a module!



### 1. Exporting from a Module

#### Named Export

```
// math.ts
export const PI = 3.14;
export function add(a: number, b: number): number {
  return a + b;
}
```

#### Default Export

```
// logger.ts
export default function log(message: string): void {
  console.log(message);
}
```

### 2. Importing in Another Module

```
// main.ts

// Named Imports
import { PI, add } from './math';
console.log(PI);
console.log(add(2, 3));

// Default Import
import log from './logger';
log('Hello TypeScript!');
```

Use named import or default import as needed.



### 3. File Organization Example

```
my-project/
├── src/
│   ├── utils/
│   │   ├── math.ts
│   │   └── logger.ts
│   ├── models/
│   │   └── user.ts
│   └── main.ts
├── index.html
├── tsconfig.json
└── package.json
```

← Root folder  
← Source code  
← Utility functions  
← Logging module  
← Models / Types  
← Entry point  
← HTML file  
← TypeScript config  
← Project metadata

#### Tips

- ★ Use meaningful file and folder names.
- ★ Keep related code in the same folder.
- ★ Use **index.ts** files to re-export modules (barrel files).



### 4. Barrel File (index.ts)

```
// utils/index.ts
export * from './math';
export * from './logger';

// main.ts
import { add, PI } from './utils';
import log from './utils';
```

Barrel files help us import from a single place!



### 5. tsconfig.json (Important Settings)

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "CommonJS",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "esModuleInterop": true
  }
}
```

This file helps TypeScript know how to compile your project.



### Key Takeaways

- ✓ Modules help us write clean, reusable and maintainable code.
- ✓ Use **export** to share code.
- ✓ Use **import** to use code from other files.
- ✓ Organize files and folders properly for scalability.



### Practice Time! ✨

- ① Create a module **math.ts** with functions (add, subtract).
- ② Export them and import in **main.ts**.
- ③ Create a **logger.ts** with a default export.
- ④ Organize files as shown in the example structure.



★ Good file organization today, easier maintenance tomorrow! ❤️



TS

# TypeScript

BASIC (10 DAYS)

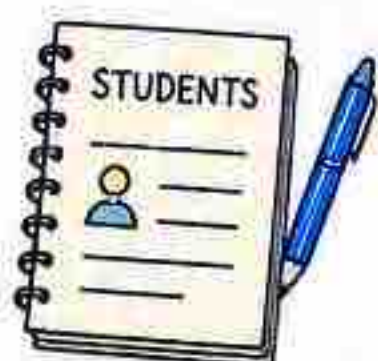
## DAY 10: Mini Project - Student Management System

Let's build a simple **Student Management System** using TypeScript!

### 1. Project Overview

We will create a console-based application to perform **CRUD** operations on students.

- ✓ Add new students
- ✓ View all students
- ✓ Search student by ID
- ✓ Update student details
- ✓ Delete student



### 2. Features

- ★ Type-safe data using interfaces
- ★ In-memory storage using an array
- ★ Menu-driven console application
- ★ Clean, modular code using functions
- ★ Easy to extend and maintain



### 3. Data Model (TypeScript Interfaces)

```
interface Student {  
  id: number;  
  name: string;  
  age: number;  
  grade: string;  
  email: string;  
}
```

Using an interface ensures type safety and consistency!



### 4. Core Functionalities (Functions)

```
function addStudent(students: Student[], student: Student): void  
  // Add a new student  
function getAllStudents(students: Student[]): void  
  // Display all students  
function getStudentById(students: Student[], id: number): void  
  // Search student by ID  
function updateStudent(students: Student[], id: number, updated: Partial<Student>): void  
  // Update student details  
function deleteStudent(students: Student[], id: number): void  
  // Delete student
```

### 5. Sample Data (Optional Start)

```
let students: Student[] = [  
  { id: 1, name: "Alice", age: 20,  
    grade: "A", email: "alice@mail.com" },  
  { id: 2, name: "Bob", age: 21,  
    grade: "B", email: "bob@mail.com" }  
];
```

Stored in memory using an array.



### 6. Menu (Console Interface)

===== Student Management System =====

1. Add Student
2. View All Students
3. Search Student by ID
4. Update Student
5. Delete Student
6. Exit

Enter your choice: \_\_

User selects an option and we call the corresponding function.

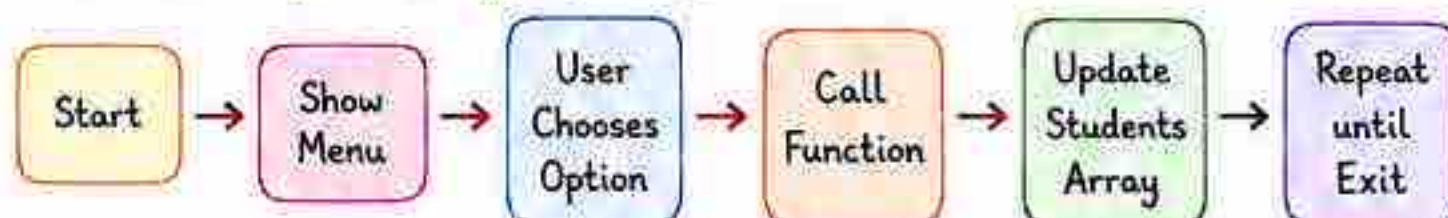


### 7. File Structure Suggestion

```
student-management/  
├── src/  
│   ├── models/  
│   │   └── student.ts  
│   ├── services/  
│   │   └── studentService.ts  
│   ├── utils/  
│   │   ├── menu.ts  
│   │   └── index.ts  
├── tsconfig.json  
└── package.json
```

← Root folder  
← Interfaces / Types  
← Business logic  
← Menu & helpers  
← Entry point  
← TS Config  
← Project metadata

### 8. Example Usage (Flow)



### 9. What I Learned

- ✓ Applied TypeScript concepts in a real mini project.
- ✓ Used interfaces, functions, arrays, and impdules.
- ✓ Practiced clean code and file organization.
- ✓ Built a foundation for larger applications!



### Key Takeaways

- ✓ TypeScript makes code more reliable and predictable.
- ✓ Good structure and modular code are important.
- ✓ Small projects build strong coding habits!



## Congratulations!

You completed all 10 days of TypeScript Basics!  
Keep coding and keep learning!

